Module 10

Topics: More on reporting/output, Exporting data to many different types of files

Files to download: tagsetex.sas7bdat
SAS Library files: SASHELP.CLASS, SASHELP.IRIS

10.1 – Exporting Data

SAS can export many different types of files. Data can be exported with the menu based Wizard (similar to the Import Wizard we used in Module 2) or with the code based PROC EXPORT procedure. The basic syntax for PROC EXPORT is below. See SAS documentation for a list of all valid file extensions (http://support.sas.com/documentation/cdl/en/acpcref/63184/HTML/default/viewer.htm#a003102702.htm).

```
PROC EXPORT
DATA=<libref.SAS data-set (SAS data-set-option(s))>
DBMS=<data-source-identifier>
LABEL
OUTFILE=<'filename'>|OUTTABLE='tablename'
REPLACE;
```

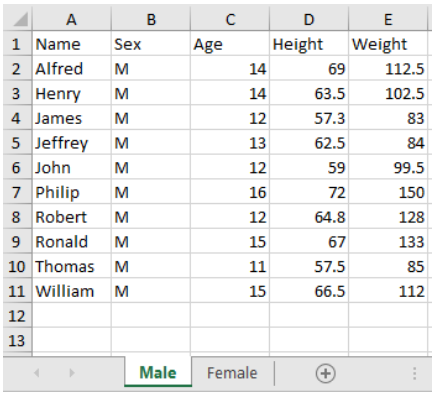
Let's do an example with one of the most common export types, Microsoft Excel. The following code can be used to export an Excel file from the SASHELP library dataset CLASS. Note that you will need to change the part of the pathway in blue to point to a location that is on your specific computer or network drive. I am naming my excel file "class" but you could change that portion in yellow to something else if you want – you do need to keep the ".xlsx" extension at the end.

```
PROC EXPORT DATA= sashelp.class
    OUTFILE= 'H:\AEPI537D\class.xlsx'
    DBMS=EXCEL REPLACE;
    SHEET="class";
RUN;
```

You can also export different SAS datasets or subsets of a SAS dataset to multiple sheets in a single Excel file. Below we will use 2 PROC EXPORT steps to put the Male students from the SASHELP.CLASS dataset in one sheet and the female students in another sheet in the same file. Note that you will need to change the part of the pathway in blue to point to a location that is on your specific computer or network drive. I am calling my Excel file "classmulti" but again you can change the name in yellow to anything you want. When exporting to the same file you MUST be certain you name the file the same way in each proc export – if I made a typo in the yellow of one of these steps below it would result in 2 different files being created.

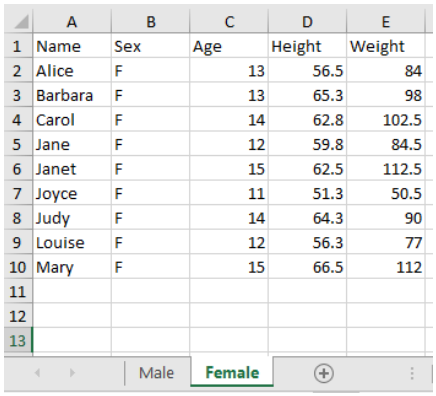
```
proc export data=sashelp.class(where=(sex='M'))
            outfile="H:\AEPI537D\classmulti.xlsx"
            dbms=excel replace label;
                 sheet="Male";
run;
```

```
proc export data=sashelp.class(where=(sex='F'))
             outfile="H:\AEPI537D\classmulti.xlsx"
             dbms=excel replace label;
                  sheet="Female";
run;
```



	A	B	C	D	E
1	Name	Sex	Age	Height	Weight
2	Alfred	M	14	69	112.5
3	Henry	M	14	63.5	102.5
4	James	M	12	57.3	83
5	Jeffrey	M	13	62.5	84
6	John	M	12	59	99.5
7	Philip	M	16	72	150
8	Robert	M	12	64.8	128
9	Ronald	M	15	67	133
10	Thomas	M	11	57.5	85
11	William	M	15	66.5	112
12					
13					

Male | Female



	A	B	C	D	E
1	Name	Sex	Age	Height	Weight
2	Alice	F	13	56.5	84
3	Barbara	F	13	65.3	98
4	Carol	F	14	62.8	102.5
5	Jane	F	12	59.8	84.5
6	Janet	F	15	62.5	112.5
7	Joyce	F	11	51.3	50.5
8	Judy	F	14	64.3	90
9	Louise	F	12	56.3	77
10	Mary	F	15	66.5	112
11					
12					
13					

Male | Female

Sometimes TAGSETS can also be helpful when creating Excel exports. This is a way to quickly get formatted output from a procedure such as a PROC PRINT into an Excel file.

First let's apply some temporary formats to our dataset TAGSETEX.

```
PROC FORMAT;
      VALUE YNF
            0='NO'
            1='YES';
      VALUE TYPE
            1='Rarely'
            2='Sometimes'
            3='Always';
      VALUE GROUP
            1='Group A'
            2='Group B'
            3='Group C';
RUN;

data tagsetex_fmt;
      set aepi537d.tagsetex;
            format var1 ynf. var2 type. var3 group.;
run;
```

In SAS our TAGSETEX_FMT dataset now looks like this when we open the Viewtable.

	study_id	var1	var2	var3
1	1001	YES	Rarely	Group B
2	1002	YES	Sometimes	Group C
3	1003	YES	Rarely	Group A
4	1004	NO	Always	Group C
5	1005	NO	Rarely	Group A
6	1006	YES	Always	Group A
7	1007	NO	Sometimes	Group B
8	1008	YES	Sometimes	Group B
9	1009	NO	Sometimes	Group A
10	1010	YES	Always	Group B

But if we do an Excel export of the TAGSETEX_FMT dataset it would look like this. Without a code book I have no idea what the values mean since I no longer have the SAS formats.

```
PROC EXPORT DATA= tagsetex_fmt
    OUTFILE= 'H:\AEPI537D\tagsetex_fmt.xlsx'
    DBMS=EXCEL REPLACE;
    SHEET="tagsetex_fmt";
RUN;
```

	A	B	C	D
1	study_id	var1	var2	var3
2	1001	1	1	2
3	1002	1	2	3
4	1003	1	1	1
5	1004	0	3	3
6	1005	0	1	1
7	1006	1	3	1
8	1007	0	2	2
9	1008	1	2	2
10	1009	0	2	1
11	1010	1	3	2

TAGSETS allows you to create an Excel style export from a procedure such as proc print and see the formatted data values. I use this often when making quick output to share with people that are not as familiar with the data. Note that this is an XML file – it will open with Excel but needs to be saved as an Excel file type if that is the goal.

```
ods Tagsets.ExcelXP file='H:\AEPI537D\tagsetex_fmt.xml';
      proc print data=tagsetex_fmt noobs; run;
ods Tagsets.ExcelXP close;
```

	A	B	C	D
1	study_id	var1	var2	var3
2	1001	YES	Rarely	Group B
3	1002	YES	Sometimes	Group C
4	1003	YES	Rarely	Group A
5	1004	NO	Always	Group C
6	1005	NO	Rarely	Group A
7	1006	YES	Always	Group A
8	1007	NO	Sometimes	Group B
9	1008	YES	Sometimes	Group B
10	1009	NO	Sometimes	Group A
11	1010	YES	Always	Group B

TAGSETS has tons of style options that can be specified. You can add gridlines, filters, freeze headers, etc. See SAS help for a list of all the defaults and supported options (https://support.sas.com/rnd/base/ods/odsmarkup/excelxp_help.html).

10.2 – Proc Report

PROC REPORT is a very useful procedure to generate professional looking reports. There are entire books written about PROC REPORT customization (and many articles can be found online) and it takes quite some time to master.

In general PROC REPORT typically looks something like this. COLUMNS defines the columns and their order, DEFINE determines how each variable is used, COMPUTE blocks allow for calculations, and BREAK/RBREAK puts physical breaks in the report.

PROC REPORT data=SAS-data-set *options* ;
    COLUMNS variable_1 …. variable_*n*;
    DEFINE variable_1;
    DEFINE variable_2;
    …..
    DEFINE variable_*n*;

    COMPUTE ***blocks***
    BREAK …..;
    RBREAK ….;
RUN;

PROC REPORT has endless options. See SAS help for more information about PROC REPORT and the options that can be specified (http://support.sas.com/documentation/cdl/en/proc/65145/HTML/default/viewer.htm#p1g25rcvj5mgxln1pdhjqubedzc9.htm).

Getting started with PROC REPORT can be a bit overwhelming. The following SAS conference proceeding article contains a great overview of PROC REPORT using the SASHELP.CLASS dataset you are

already familiar with from previous exercises. The link to the article is below, and the article is included as a PDF with this module as well.

https://www.mwsug.org/proceedings/2010/tutorials/MWSUG-2010-109.pdf

I recommend walking through each step of the article and running the code to see the output. To make this a bit easier I have created a SAS program for you with all of code for each "trick". The program is called "SAS Code for Mod 10 MWSIG article" and it can be found with the materials for this module.

Tips:

- Another powerful tool for creating SAS output is PROC TABULATE. Most tasks can be accomplished with both procedures, but you will find that some things are easier with one or the other. The popular opinion online is that Tabulate is better for cross-tabulation or hierarchical groupings but that Report has the edge for everything else. I would recommend checking out some articles for both procedures and find the one that works best for you – the put the time into really learning about the options and mastering it!
- Another handy output related tip I use quite often for large datasets is sorting a PROC CONTENTS by the variable position. By default, PROC CONTENTS will list variables in alphabetical order rather than the order they appear in the dataset (in the Viewtable). You can use some built in features of PROC CONTENTS to sort by the variable order and generate a clean list of variables as they appear.

  In the step below, I am creating an output dataset called "check_contents" from the sashelp.class dataset. I am using the noprint option to avoid displaying the proc contents output as I only want the dataset. This dataset gives me the information from the proc contents in variables including the variable name, label, length, and number (position).

```
proc contents data= sashelp.class out=check_contents noprint; run;
```

  Next, I create a dataset called "check_contents2" from "check_contents" that keeps only the name, label, and position of the variable in the dataset (varnum).

```
data check_contents2; set check_contents; keep varnum name label; run;
```

  I can then sort by the variable position and print out a list of my variables as they appear in the dataset.

```
proc sort data= check_contents2; by varnum; run;

proc print data= check_contents2 noobs; var name label; run;
```

MOD 10: USE SAS COMMENTS TO NUMBER YOUR WORK.

Using the pre-loaded SAS dataset in the SASHELP library called IRIS (SET SASHELP.IRIS):

1. Use PROC EXPORT steps to export the SASHELP.IRIS dataset into an Excel file with a DIFFERENT worksheet for each iris species (variable = Species). Note that you will need to figure out if the variable is character or numeric before you can write the where statements .

2. Also using the SASHELP.IRIS dataset, use PROC REPORT to generate a table with the **mean** petal width (variable = PetalWidth) for **each** iris species (variable = Species). Your output should look something like this:

Iris Species	Petal Width
Setosa	2.5
Versicolor	13.3
Virginica	20.3